SKKU 2024 Challenge: Heisenberg Spin Glass

Donghun Jung

November 18, 2024

Part I : A first look at the Heisenberg spin glass

How many symmetry classes are there?

There are 11 distinct symmetry classes, each characterized by unique eigen-spectra. I conducted eigen-decomposition for all 64 possible combinations of the coefficients J_{ij}.

The following code generates the Heisenberg Hamiltonian $H_{heisenberg}$:

```
def __reset__(self):
    qubit_idx = [idx for idx in range(self.num_qubits)]
    combinations = itertools.combinations(qubit_idx, 2)

    H_z = np.zeros([2**self.num_qubits, 2**self.num_qubits]
                   , dtype=np.complex64)
    if self.coefficients2 is not None:
        for i in range(self.num_qubits):
            temp = [self.I for _ in range(self.num_qubits)]
            temp[i] = self.Pauli_Z
            H_z += self.coefficients2[i]*self._Kron(temp)

    H_heisenberg = np.zeros([2**self.num_qubits,
                             2**self.num_qubits]
                             , dtype=np.complex64)
    for coefficient, pair_idx
      in zip(self.coefficients1, combinations):
        H_heisenberg += coefficient * self._H_ij(pair_idx)
    self.H_sg = H_heisenberg + H_z
```

In this example, `self.coefficients2` is set to `None`. Within the `for` loop, each pair of qubit indices (i, j) runs through unique combinations in the following sequence: $(0,0)$, $(0,1)$, $(0,2)$, $(1,2)$, $(1,3)$, $(2,3)$. For next section, `for` loop with `self.coefficients2`, qubit indices i runs from 0

to 3. Additionally, the `_Kron` method is implemented to ensure the Qiskit bit-ordering convention.

```
def _Kron(self, array):
    prod = np.array(1)
    if len(array) == 0:
        return prod
    else:
        for element in array:
            prod = np.kron(element, prod)
        return prod
```

Two models are classified within the same symmetry class if their eigenspectra are identical. Here, the eigenspectrum of a Hamiltonian represents the set of its eigenvalues. To numerically determine whether two eigenspectra are equivalent, I used the `np.allclose` function.

```
symmetry_classes = {}
for coefficient_combination
  in itertools.product([1, -1], repeat=6):

    eigenspectrum = H_sg(coefficient_combination, [0, 0, 0, 0])
    symmetry_classes[coefficient_combination] = eigenspectrum

classified_groups = {}
unique_eigenspectra = {}
class_count = 0

for combination, spectrum in symmetry_classes.items():
  is_duplicate = False
  for class_id, class_spectrum in unique_eigenspectra.items():
    if np.allclose(class_spectrum, spectrum._eigenSpectrum()):
      is_duplicate = True
      classified_groups[class_id].append(combination)
      break

  if is_duplicate:
    continue
  else:
    class_count += 1
    classified_groups[class_count] =[combination]
    unique_eigenspectra[class_count]=spectrum._eigenSpectrum()
```

Detailed information on eigenspectra and coefficient values can be found in the next questions.

For each symmetry class, provide one set of J_{ij} coefficients for a model in that symmetry class.

The Heisenberg Hamiltonian $\mathcal{H}_{\text{heisenberg}}$ is defined as:

$$\mathcal{H}_{\text{heisenberg}} = \sum_{(i,j)} J_{i,j} H_{ij} \tag{1}$$

where

$$H_{ij} = \sigma_x^i \sigma_x^j + \sigma_y^i \sigma_y^j + \sigma_z^i \sigma_z^j \tag{2}$$

The interaction is classified as ferromagnetic when $J_{ij} = 1$ and antiferromagnetic when $J_{ij} = -1$. This Hamiltonian possesses intrinsic interchange symmetry, meaning that under an interchange of qubits i and j, both the eigenvalues and eigenstates of the Hamiltonian remain unchanged. Consequently, all elements within the same symmetry class can be generated through such interchange operations.

In the diagram below, blue edges represent ferromagnetic interactions ($J_{ij} = 1$), and red edges represent anti-ferromagnetic interactions($J_{ij} = -1$).

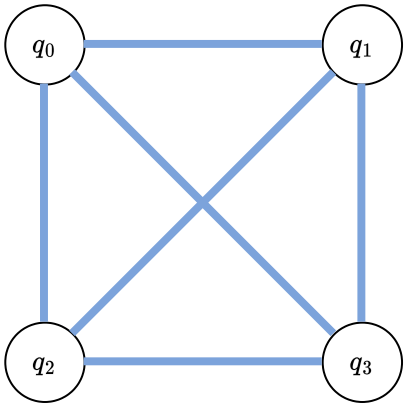

Figure 1: Visual representation of a four-qubit system with ferromagnetic interactions. Each blue edge corresponds to a ferromagnetic interaction ($J_{ij} = 1$) between qubits $q_i$and q_j. The system belongs to Class 1, where all interactions are ferromagnetic.

Class 1. The whole interactions are ferromagnetic, making $J_{ij} = 1$ for every pair of qubits. Since the symmetry class remains invariant under interchange operations, no other elements can exist within this class.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	1	1	1	1

Table 1: Coefficients J_{ij} for Class 1 of the Heisenberg Hamiltonian.

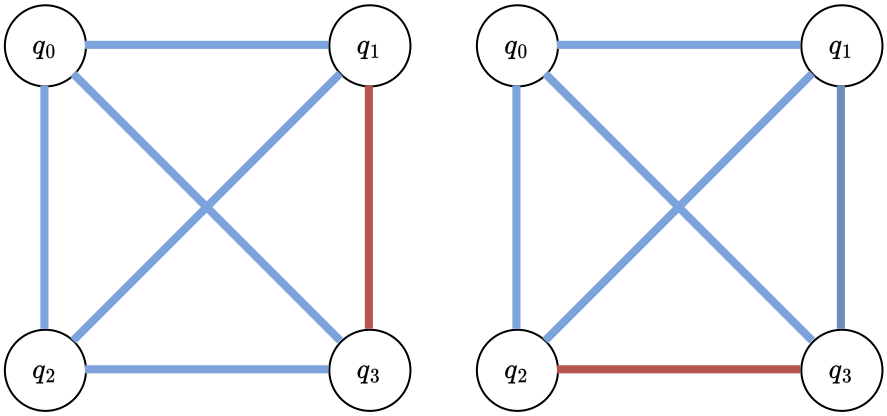

Figure 2: Two examples of models in Class 2. Left: Model with coefficients $(1, 1, 1, 1, -1, 1)$. Right: Model with coefficients $(1, 1, 1, 1, 1, -1)$, derived via qubit interchange.

Class 2. The following table provides the J_{ij} coefficients for Class 2 of the Heisenberg Hamiltonian, where exactly one interaction is anti-ferromagnetic. The figure below illustrates two examples of models in Class 2. In the left figure, the coefficients are $(1, 1, 1, 1, -1, 1)$, with a single anti-ferromagnetic interaction denoted by a red edge. By applying an interchange operation on qubits q_1 and q_2, we derive the model shown on the right, with coefficients $(1, 1, 1, 1, 1, -1)$. Through additional interchange operations, the remaining models listed in Table 2 can be generated.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	1	1	1	-1
1	1	1	1	-1	1
1	1	1	-1	1	1
1	1	-1	1	1	1
1	-1	1	1	1	1
-1	1	1	1	1	1

Table 2: Coefficients J_{ij} for Class 2 of the Heisenberg Hamiltonian.

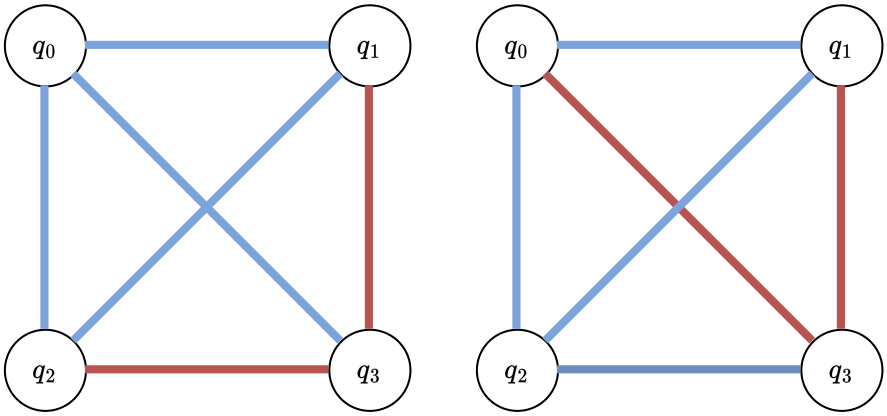

Figure 3: Two examples of models in Class 3. Left: Model with coefficients $(1, 1, 1, 1, -1, -1)$, where qubit q_3 has two anti-ferromagnetic interactions denoted by red edges. Right: Model with coefficients $(1, 1, -1, 1, -1, 1)$, derived by applying an interchange operation on qubits q_0 and q_2.

Class 3. The following table provides the J_{ij} coefficients for Class 3 of the Heisenberg Hamiltonian, characterized by exactly two anti-ferromagnetic interactions. The figure illustrates two examples of models in Class 3. In the left figure, the coefficients are $(1, 1, 1, 1, -1, -1)$, with qubit q_3 having two anti-ferromagnetic interactions, represented by red edges. By applying an interchange operation on qubits q_0 and q_2, we derive the model shown on the right, with coefficients $(1, 1, -1, 1, -1, 1)$. Through additional interchange operations, the remaining models listed in Table 3 can be generated.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	1	1	−1	−1
1	1	1	−1	1	−1
1	1	1	−1	−1	1
1	1	−1	1	1	−1
1	1	−1	1	−1	1
1	−1	1	1	1	−1
1	−1	1	−1	1	1
1	−1	−1	1	1	1
−1	1	1	1	−1	1
−1	1	1	−1	1	1
−1	1	−1	1	1	1
−1	−1	1	1	1	1

Table 3: Coefficients J_{ij} for Class 3 of the Heisenberg Hamiltonian.

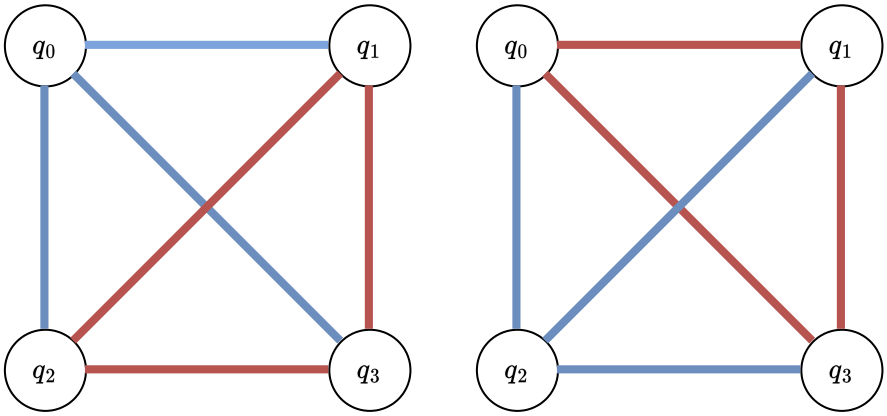

Figure 4: Two examples of models in Class 4. Left: Model with coefficients $(1, 1, 1, -1, -1, -1)$, where three qubits each have two anti-ferromagnetic interactions, denoted by red edges, and the remaining qubit interacts ferromagnetically with all neighbors. Right: Model with coefficients $(-1, 1, -1, 1, -1, 1)$, derived by applying an interchange operation on qubits q_0 and q_2.

Class 4. Three interaction terms are anti-ferromagnetic. As shown in the figure, three qubits have two anti-ferromagnetic interactions, while the remaining qubit has only ferromagnetic interactions. By applying interchange operations, four elements within the same symmetry class can be derived. For example, starting from the left model in the figure with coefficients $(1, 1, 1, -1, -1, -1)$, we can derive the right model with coefficients $(-1, 1, -1, 1, -1, 1)$ by applying an interchange operation on qubits q_0 and q_2. The other models can be obtained by applying one or multiple interchange operations.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	1	−1	−1	−1
1	−1	−1	1	1	−1
−1	1	−1	1	−1	1
−1	−1	1	−1	1	1

Table 4: Coefficients J_{ij} for Class 4 of the Heisenberg Hamiltonian.

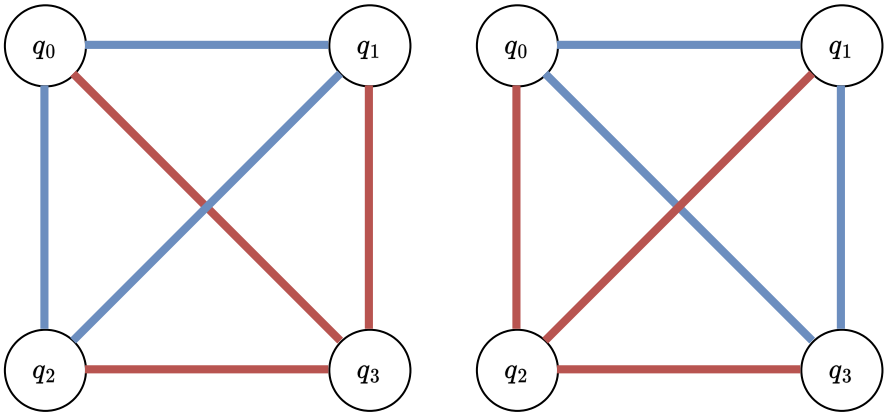

Figure 5: Two examples of models in Class 5. Left: Model with coefficients $(1, 1, -1, 1, -1, -1)$, where qubit q_3 has only anti-ferromagnetic interactions. Right: Model with coefficients $(1, -1, 1, 1, -1, -1)$, derived by applying an interchange operation on qubits q_2 and q_3.

Class 5. Three interaction terms are anti-ferromagnetic. As shown in the figure, one qubit has only anti-ferromagnetic interactions. By applying interchange operations, four elements within the same symmetry class can be derived. For example, starting from the left model in the figure with coefficients $(1, 1, -1, 1, -1, -1)$, we can derive the right model with coefficients $(1, -1, 1, 1, -1, -1)$ by applying an interchange operation on qubits q_2 and q_3. The other models can be obtained by applying one or multiple interchange operations. Note that this symmetry class can also be derived by applying a sign-flip operation to the elements of symmetry Class 4.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	−1	1	−1	−1
1	−1	1	−1	1	−1
−1	1	1	−1	−1	1
−1	−1	−1	1	1	1

Table 5: Coefficients J_{ij} for Class 5 of the Heisenberg Hamiltonian.

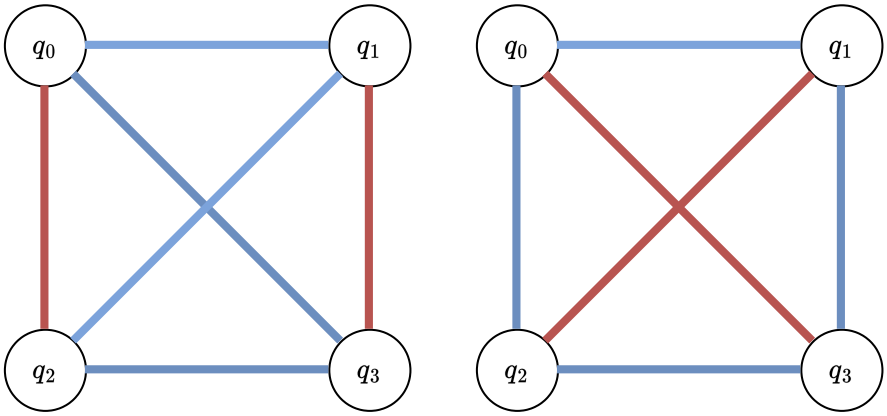

Figure 6: Two examples of models in Class 6. Left: Model with coefficients $(1, -1, 1, 1, -1, 1)$, where each qubit has one anti-ferromagnetic interaction. Right: Model with coefficients $(1, 1, -1, -1, 1, 1)$, derived by applying an interchange operation on qubits q_0 and q_1.

Class 6. Two interaction terms are anti-ferromagnetic. As shown in the figure, each qubit has exactly one anti-ferromagnetic interaction, unlike the Class 2 symmetry group, where one qubit has two anti-ferromagnetic interactions. By applying interchange operations, three elements within the same symmetry class can be derived. For example, starting from the left model in the figure with coefficients $(1, -1, 1, 1, -1, 1)$, we can derive the right model with coefficients $(1, 1, -1, -1, 1, 1)$ by applying an interchange operation on qubits q_0 and q_1. The remaining model can be derived by applying one or multiple interchange operations.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	−1	−1	1	1
1	−1	1	1	−1	1
−1	1	1	1	1	−1

Table 6: Coefficients J_{ij} for Class 6 of the Heisenberg Hamiltonian.

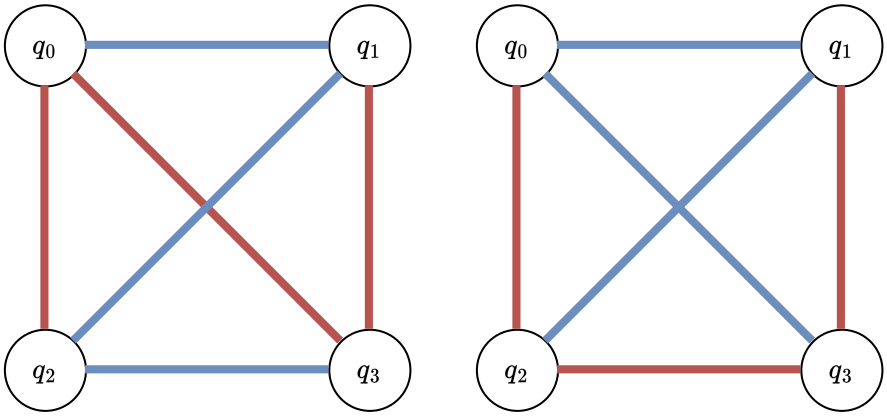

Figure 7: Two examples of models in Class 7. Left: Model with coefficients $(1, -1, -1, 1, -1, 1)$, where each qubit has one or two anti-ferromagnetic interactions. Right: Model with coefficients $(1, -1, 1, 1, -1, -1)$, derived by applying an interchange operation on qubits q_0 and q_2. The symmetry class also allows sign-flip operations, preserving the class structure.

Class 7. Three interaction terms are anti-ferromagnetic. As shown in the figure, each qubit has either one or two anti-ferromagnetic interactions. This symmetry class also possesses sign-flip symmetry, meaning that after flipping the signs of all J_{ij} coefficients, the resulting Hamiltonian remains within the same symmetry class. By applying interchange operations and sign-flip operations, 12 elements within this symmetry class can be derived. For example, starting from the left model in the figure with coefficients $(1, -1, -1, 1, -1, 1)$, we can derive the right model with coefficients $(1, -1, 1, 1, -1, -1)$ by applying an interchange operation on qubits q_0 and q_2. The other models can be obtained by applying one or multiple interchange operations and sign-flip operations.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	−1	−1	1	−1
1	1	−1	−1	−1	1
1	−1	1	1	−1	−1
1	−1	1	−1	−1	1
1	−1	−1	1	−1	1
1	−1	−1	−1	1	1
−1	1	1	1	−1	−1
−1	1	1	−1	1	−1
−1	1	−1	1	1	−1
−1	1	−1	−1	1	1
−1	−1	1	1	1	−1
−1	−1	1	1	−1	1

Table 7: Coefficients J_{ij} for Class 7 of the Heisenberg Hamiltonian.

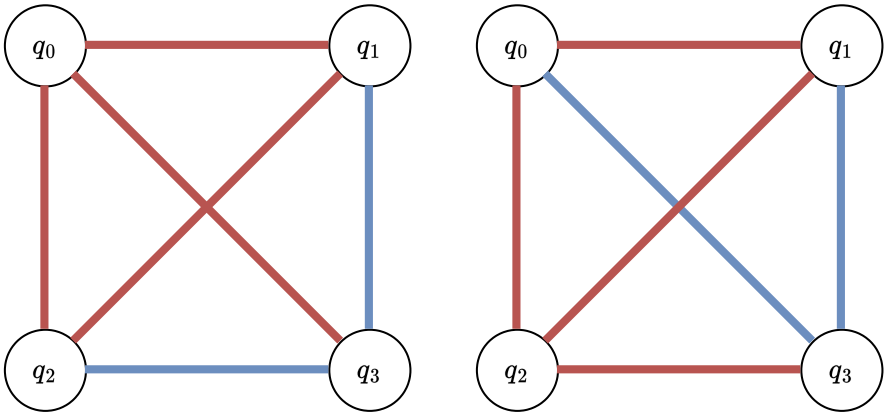

Figure 8: Two examples of models in Class 8. Left: Model with coefficients $(-1,-1,-1,-1,1,1)$, where one qubit (q_3) has two ferromagnetic interactions. Right: Model with coefficients $(-1,-1,1,-1,1,-1)$, derived by applying an interchange operation on qubits q_0 and q_2.

Class 8. From this point onward, it is convenient to focus on ferromagnetic interactions. In this symmetry class, two interaction terms are ferromagnetic. As shown in the figure, one qubit has two ferromagnetic interactions. By applying interchange operations, 12 elements within the same symmetry class can be derived. For example, starting from the left model in the figure with coefficients $(-1,-1,-1,-1,1,1)$, we can derive the right model with coefficients $(-1,-1,1,-1,1,-1)$ by applying an interchange operation on qubits q_0 and q_2. The other models can be obtained by applying one or multiple interchange operations. Note that this symmetry class can also be derived by applying a sign-flip operation to the elements of symmetry Class 3.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	−1	−1	−1	−1
1	−1	1	−1	−1	−1
1	−1	−1	1	−1	−1
1	−1	−1	−1	1	−1
−1	1	1	−1	−1	−1
−1	1	−1	1	−1	−1
−1	1	−1	−1	−1	1
−1	−1	1	−1	1	−1
−1	−1	1	−1	−1	1
−1	−1	−1	1	1	−1
−1	−1	−1	1	−1	1
−1	−1	−1	−1	1	1

Table 8: Coefficients J_{ij} for Class 8 of the Heisenberg Hamiltonian.

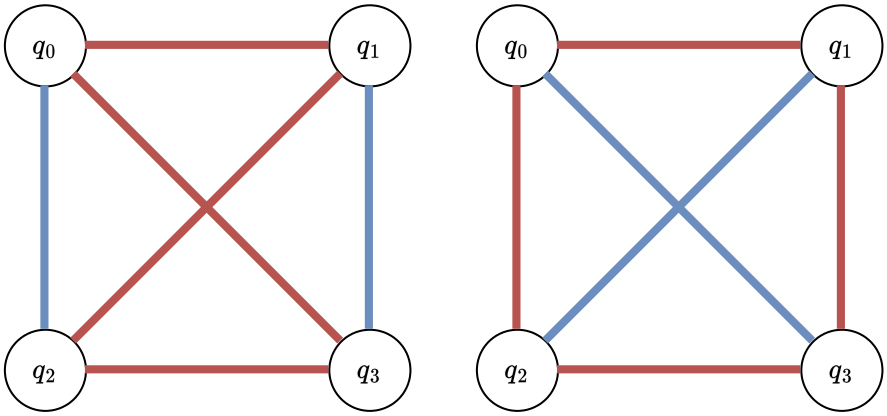

Figure 9: Two examples of models in Class 9. Left: Model with coefficients $(-1, 1, -1, -1, 1, -1)$, where each qubit has one ferromagnetic interaction. Right: Model with coefficients $(-1, -1, 1, 1, -1, -1)$, derived by applying an interchange operation on qubits q_0 and q_1.

Class 9. Two interaction terms are ferromagnetic. As shown in the figure, each qubit has exactly one anti-ferromagnetic interaction. By applying interchange operations, three elements within the same symmetry class can be derived. For example, starting from the left model in the figure with coefficients $(-1, 1, -1, -1, 1, -1)$, we can derive the right model with coefficients $(-1, -1, 1, 1, -1, -1)$ by applying an interchange operation on qubits q_0 and q_1. The remaining model can be derived by applying one or multiple interchange operations. Note that this symmetry class can also be obtained by performing a sign-flip operation on the elements of symmetry Class 6.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	−1	−1	−1	−1	1
−1	1	−1	−1	1	−1
−1	−1	1	1	−1	−1

Table 9: Coefficients J_{ij} for Class 9 of the Heisenberg Hamiltonian.

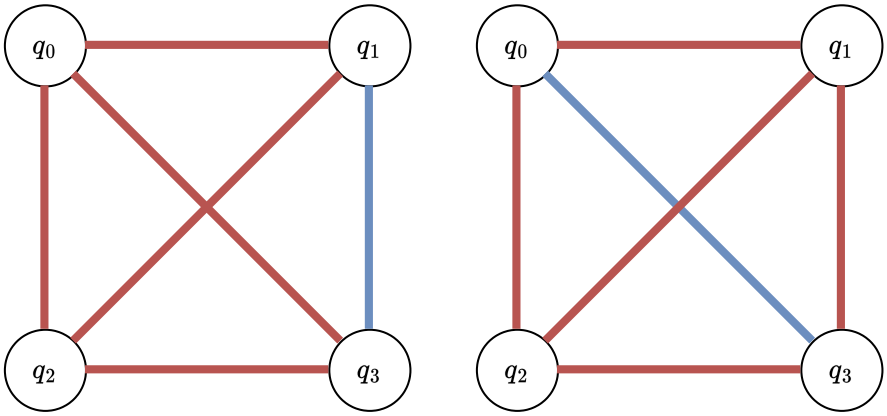

Figure 10: Two examples of models in Class 10. Left: Model with coefficients $(-1,-1,-1,-1,1,-1)$, where only one interaction term is ferromagnetic. Right: Model with coefficients $(-1,-1,1,-1,-1,-1)$, derived by applying an interchange operation on qubits q_0 and q_1.

Class 10. Only one interaction term is ferromagnetic. By applying interchange operations, six elements within the same symmetry class can be derived. For example, starting from the left model in the figure with coefficients $(-1,-1,-1,-1,1,-1)$, we can derive the right model with coefficients $(-1,-1,1,-1,-1,-1)$ by applying an interchange operation on qubits q_0 and q_1. The remaining models can be obtained by applying one or multiple interchange operations. Note that this symmetry class can also be derived by applying a sign-flip operation to the elements of symmetry Class 2.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	−1	−1	−1	−1	−1
−1	1	−1	−1	−1	−1
−1	−1	1	−1	−1	−1
−1	−1	−1	1	−1	−1
−1	−1	−1	−1	1	−1
−1	−1	−1	−1	−1	1

Table 10: Coefficients J_{ij} for Class 10 of the Heisenberg Hamiltonian.

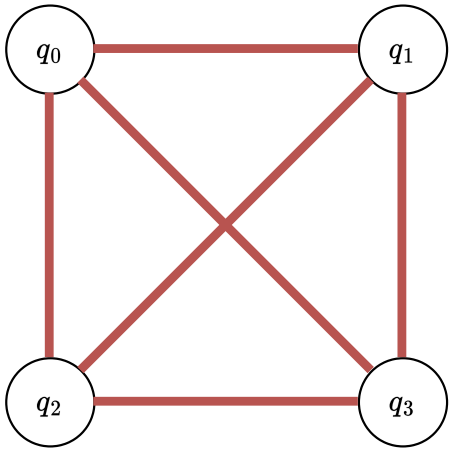

Figure 11: The unique model in Class 11, with coefficients $(-1, -1, -1, -1, -1, -1)$. All interactions are anti-ferromagnetic, and no additional elements exist within this class, even under interchange operations. This class can also be derived through sign-flip operations on Class 1 models.

Class 11. All interactions in this class are anti-ferromagnetic. There are no additional possible elements, even when applying interchange operations. This symmetry class can also be derived by applying a sign-flip operation to the elements of symmetry Class 1.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
-1	-1	-1	-1	-1	-1

Table 11: Coefficient J_{ij} for Class 11 of the Heisenberg Hamiltonian.

What is the eigenspectrum of each symmetry class?

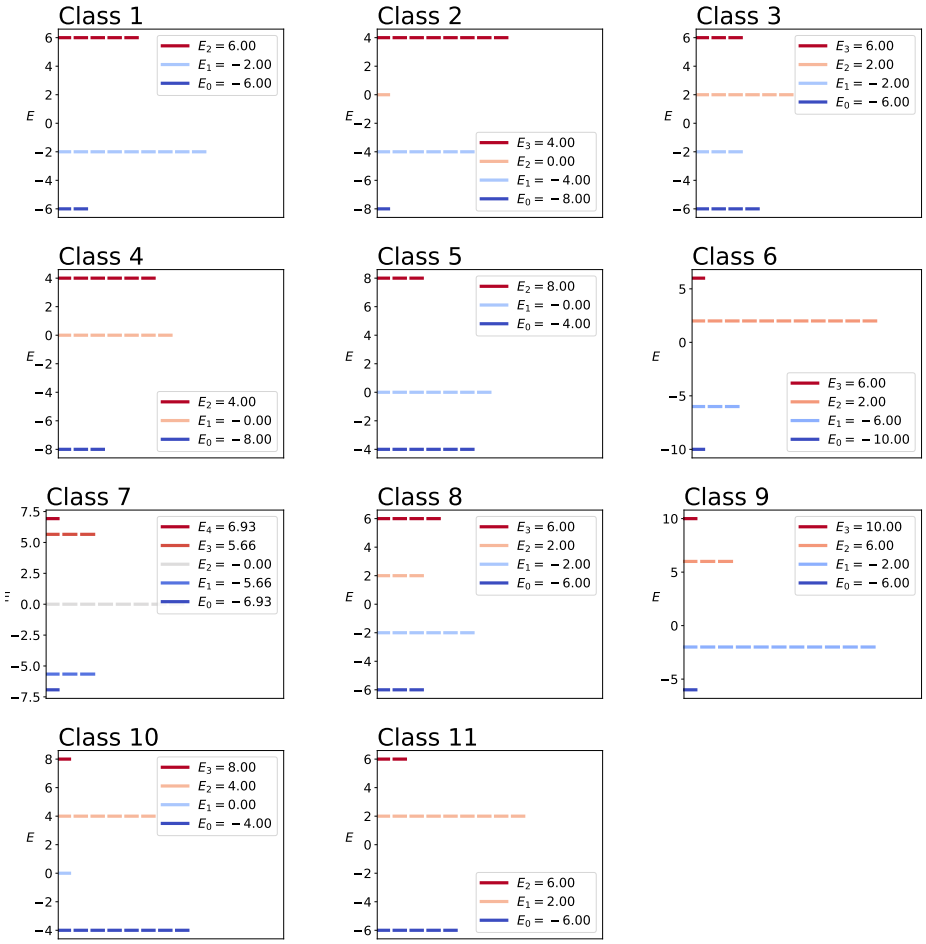

Figure 12: Eigenspectra of the 11 symmetry classes. Higher energy levels are shown in red tones, while lower energy levels are shown in blue tones. The number of cells at each energy level represents its degeneracy.

This figure presents the eigenspectra for each of the 11 symmetry classes. The energy levels are represented by horizontal lines, with color indicating their relative magnitude: higher energy levels are shown in red tones, while lower energy levels are shown in blue tones. The eigenvalues are labeled as $E_0 < E_1 < E_2 < \cdots$, and the number of cells at each energy level corresponds to its degeneracy. An important feature is the energy inversion correspondence between certain classes:

- Class 1 and Class 11 have eigenspectra that are related by flipping the signs of all energy levels.

- Class 2 and Class 10 have eigenspectra that are related by flipping the signs of all energy levels.
- Class 3 and Class 8 have eigenspectra that are related by flipping the signs of all energy levels.
- Class 4 and Class 5 have eigenspectra that are related by flipping the signs of all energy levels.
- Class 6 and Class 9 have eigenspectra that are related by flipping the signs of all energy levels.

Do any of the symmetry classes have an eigenspectrum that is sign invariant? In other words, if you change the sign of each eigenvalue, does the overall set of eigenvalues remain unchanged?

As shown in Figure 12, the eigenspectrum of Class 7 exhibits sign-flip invariance. This occurs because the Hamiltonian of Class 7 remains within Class 7 even after a sign-flip operation is applied to all its eigenvalues.

Part 2 : Adiabatic evolution to the ground state

Plot the eigenspectra as a function of s for each symmetry class.

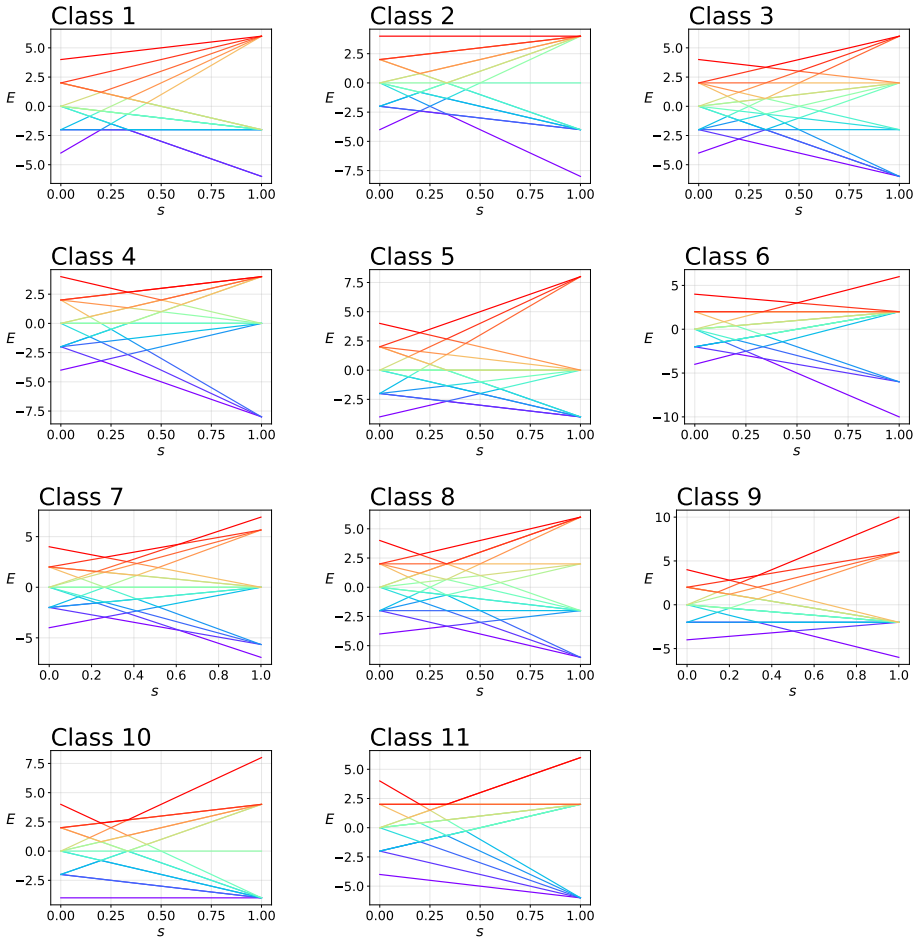

Figure 13: Evolution of eigenvalues as a function of the interpolation parameter s for the 11 symmetry classes. Red-tone lines represent higher energy eigenvalues, while violet-tone lines correspond to lower energy levels, including the ground state.

The figure illustrates the evolution of eigenvalues for each of the 11 symmetry classes as a function of the interpolation parameter s, which varies from 0 to 1. The horizontal axis represents s, while the vertical axis corresponds to energy eigenvalues. Red-tone lines indicate higher energy eigenvalues, whereas violet-tone lines denote lower energy levels, including the ground state. As s progresses from 0 (the ground state of the mixer Hamilto-

nian H_m) to 1 (the Heisenberg spin glass Hamiltonian H_{sg}), the eigenvalues shift dynamically. The mixer Hamiltonian is written as:

$$H_m = -\sum_{i=0}^{3} \sigma_x^i \tag{3}$$

Notably, in several symmetry classes, the ground state crosses with higher energy eigenstates at intermediate s values. These crossings present critical points where the adiabatic algorithm may fail, as they allow the probability to leak from the ground state into excited states, undermining the system's adiabatic evolution.

Plot the eigenspectrum for H_{sg} as a function of s.

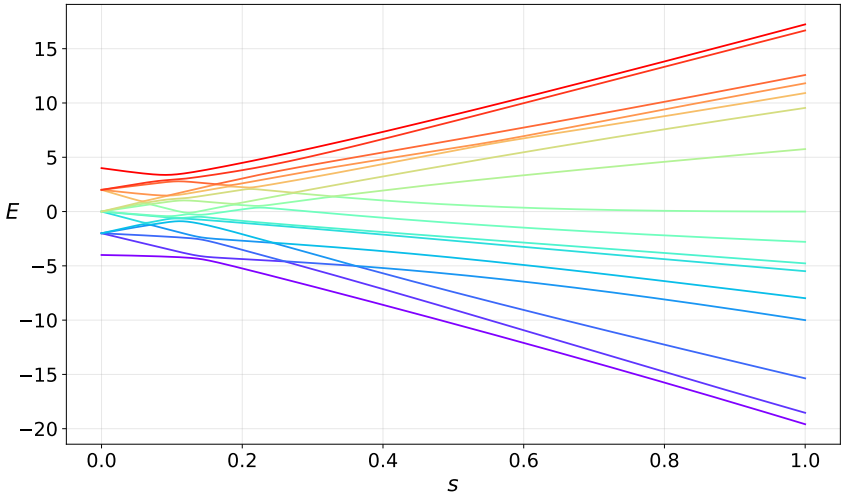

Figure 14: Evolution of eigenvalues for the modified Heisenberg spin glass Hamiltonian H_{sg} during adiabatic evolution. The horizontal axis represents the interpolation parameter s, while the vertical axis shows energy eigenvalues. Red-tone lines denote higher energy eigenvalues, and violet-tone lines denote lower energy eigenvalues, including the ground state. Notably, the ground state eigenvalue does not cross with any higher energy eigenvalues, ensuring adiabatic evolution without transitions to excited states.

Figure 14 shows the evolution of the eigenvalues for the modified Heisenberg spin glass Hamiltonian H_{sg} during adiabatic evolution, as the interpolation parameter s varies from 0 to 1. The horizontal axis represents s, while the vertical axis represents the energy eigenvalues. Red-tone lines correspond to higher energy eigenvalues, while violet-tone lines represent lower energy eigenvalues, including the ground state. A key observation is that during the adiabatic evolution, the ground state eigenvalue (lowest violet

line) does not cross with any higher energy eigenvalues. This absence of level crossings ensures that the system can reliably remain in the ground state throughout the adiabatic process.

Write a program to carry out adiabatic evolution, starting from the pure mixer Hamiltonian and ending with H_{sg}. Two parameters govern this program: the time τ over which the adiabatic evolution takes place and the number of time steps N.

The following program carries out adiabatic evolution, starting from the pure mixer Hamiltonian and ending with H_{sg}. Two parameters govern the evolution: τ, the total evolution time, and N, the number of time steps.

```
def AdiabaticEvolution(self, tau, N):
  time_step = np.linspace(0, tau, N)
  dt = time_step[1] - time_step[0]

  ground_state = np.ones(2**self.num_qubits)
                /np.sqrt(2**self.num_qubits)
  for t in time_step[1:]:
    H = self.Propagator_Hamiltonian(t/tau)
    propagator = scipy.linalg.expm(-1j*dt*H)
    ground_state = propagator @ ground_state

  return ground_state
```

Starting from the initial state $|++++\rangle$, the system evolves under the time propagator $e^{-i\Delta t H(s)}$ where the Propagator Hamiltonian is given as:

$$H(s) = (1-s)H_m + sH_{sg} \tag{4}$$

```
def Propagator_Hamiltonian(self, s):
  return (1-s)*self.H_m + s*self.H_sg
```

In order to match the target vector of sixteen probabilities by performing adiabatic evolution, what value do you need to use for τ and N? The exact target probabilities and the probabilities resulting from adiabatic evolution should match to three significant decimal digits.

The required running time for adiabatic evolution depends on the eigenspectrum of H_{sg}, particularly the energy gap, Δ, between the two lowest energy eigenvalues[3, 4]. The adiabatic evolution time τ must satisfy the following condition:

$$\tau \gg \frac{\epsilon}{\Delta^2} \tag{5}$$

where $\Delta = \min_{0 \leq s \leq 1}(E_1(s) - E_0(s))$ is the minimum energy gap between the ground state and the first excited state, and ϵ is is the largest eigenvalue of the operator, $H_{sg} - H_m$, representing the difference between the final Hamiltonian and the initial mixer Hamiltonian.

In our case, $\Delta \simeq 0.2666$ and $\epsilon \simeq 17.79$, then, we have $\tau \simeq 250.4$. After further tuning, I found that using $\tau = 1000$ and $N = 2000$(with time step size dt determined accordingly) produces the desired result, ensuring that the target probabilities and those obtained from adiabatic evolution match to three significant decimal places.

List out the terms in H_{sg} and H_m and, for each term, provide the gate that arises when that term is exponentiated. You can use any gate in Qiskit's standard gate set. How many single-qubit gates are needed for each time step? How many two-qubit gates are needed for each time step?

Given the Hamiltonians H_{sg} and H_m we have:

$$H_{sg} = \sum_{(i,j)} J_{i,j} H_{ij} + \sum_{i=0}^{3} h_i \sigma_z^i \tag{6}$$

$$H_m = -\sum_{i=0}^{3} \sigma_x^i \tag{7}$$

where $H_{ij} = \sigma_x^i \sigma_x^j + \sigma_y^i \sigma_y^j + \sigma_z^i \sigma_z^j$. The coefficients are:

$$\begin{array}{lll} J_{01} = -1 & J_{02} = 2 & J_{03} = 3 \\ J_{12} = -2 & J_{13} = -3 & J_{23} = -4 \end{array}$$

and

$$h_0 = 1 \qquad h_1 = -1 \qquad h_2 = 2 \qquad h_3 = 3.$$

For each time step in the adiabatic simulation, the time-dependent Hamiltonian $H(s)$ can be expanded as:

$$H(s) = (1-s)H_m + sH_{sg} \tag{8}$$
$$= H_x + H_y + H_z \tag{9}$$

where:

$$H_x(s) = s\sum_{(i,j)} J_{i,j}\sigma_x^i\sigma_x^j - (1-s)\sum_{i=0}^{3}\sigma_x^i \tag{10}$$

$$H_y(s) = s\sum_{(i,j)} J_{i,j}\sigma_y^i\sigma_y^j \tag{11}$$

$$H_z(s) = s\sum_{(i,j)} J_{i,j}\sigma_z^i\sigma_z^j + s\sum_{i=0}^{3} h_i\sigma_z^i \tag{12}$$

As the time steps are small enough, assuming that all terms in $H_x(s)$, $H_y(s)$ and $H_z(s)$ commute, we can separately exponentiate each term at each time step. For each time step (except the last), the following gates are needed:

- Single-qubit gates
  - 4 RX gates for σ_x^i terms in $H_x(s)$ except for $s = 1$
  - 4 RZ gates for σ_z^i terms in $H_z(s)$
- Two-qubit gates
  - 6 RXX gates for $\sigma_x^i\sigma_x^j$ terms in $H_x(s)$
  - 6 RYY gates for $\sigma_y^i\sigma_y^j$ terms in $H_y(s)$
  - 6 RZZ gates for $\sigma_z^i\sigma_z^j$ terms in $H_z(s)$

Therefore, for each time step, 8 single-qubit gates and 18 two-qubit gates are required. In the final time step, where RX gates are unnecessary, only 4 single-qubit gates and 18 two-qubit gates are needed.

What if the time step is not small enough? When the time step Δt is not sufficiently small, H_x, H_y, H_z are not commutable with each other. This necessitates the use of a Trotter-Suzuki decomposition to approximate the time propagator[6, 1, 5]. For a given time interval $t \sim t + \Delta t$. where total evolution time is τ, the time propagator is:

$$e^{-iH\Delta t} = e^{-i\Delta t\left(H_x\left(\frac{t+\Delta t}{\tau}\right)+H_y\left(\frac{t+\Delta t}{\tau}\right)+H_z\left(\frac{t+\Delta t}{\tau}\right)\right)} \tag{13}$$

Here $t + \Delta t$ ensures $s = 1$ at the final time step. Using t, would cause no evolution in the first step where $s = 0$ as the initialized + state is already an eigenstate of the first propagator.

To address the non-commutativity, we apply Trotter-Suzuki decomposition:

1. First-order approximation:

$$U(t)_1 = e^{-i\Delta t H_x\left(\frac{t+\Delta t}{\tau}\right)} e^{-i\Delta t H_y\left(\frac{t+\Delta t}{\tau}\right)} e^{-i\Delta t H_z\left(\frac{t+\Delta t}{\tau}\right)} \tag{14}$$

2. Second-order approximation:

$$U(t)_2 = U_1\left(\frac{t}{2}\right)^T U_1\left(\frac{t}{2}\right) \tag{15}$$

3. Fourth-order approximation:

$$U(t)_4 = U_2(\alpha t)\, U_2(\alpha t)\, U_2((1-4\alpha)t)\, U_2(\alpha t)\, U_2(\alpha t) \tag{16}$$

where $\alpha = \frac{1}{4-4^{\frac{1}{3}}}$.

Using the fourth-order Trotter-Suzuki approximation, each time step involves exponentiating H_x, H_y, and H_z.

1. Exponentiating H_x
   - 10 terms (for all steps except the last).
   - Requires 6 RXX gates and 4 RX gates
2. Exponentiating H_y
   - 6 terms.
   - Requires 6 RYY gates
3. Exponentiating H_z
   - 10 terms.
   - Requires 6 RZZ and 4 RZ gates,

For each time step in the fourth-order Trotter-Suzuki approximation, the circuit involves exponentiating the terms in H_x, H_y, and H_z. Specifically, H_x requires 6 RXX gates and 4 RX gates, H_y requires 6 RYY gates, and H_z requires 6 RZZ gates and 4 RZ gates. However, due to the structure of the Trotter-Suzuki sequence, the first and last exponentiated H_z terms overlap, allowing the omission of 6 RZZ gates and 4 RZ gates in each time step. As a result, a typical time step requires 30 RXX gates, 60 RYY gates, and 30 RZZ gates, along with 20 RX and 20 RZ gates, totaling 120 two-qubit gates and 40 single-qubit gates. For the final time step, where $s = 1$, the RX gates are no longer needed, reducing the single-qubit gate count to 20, while the two-qubit gate count remains at 120.

Assess the circuit needed to carry out the adiabatic evolution. Roughly how many single-qubit gates and two-qubit gates would be required to achieve three-digit accuracy for the sixteen probabilities characterizing the ground state? Do you think this is practical?

Figure 15: IONQ Forte Details.

The adiabatic evolution approach appears to be impractical. As discussed in the previous question, each time step requires 8 single-qubit gates and 18 two-qubit gates. To achieve three-digit accuracy for the sixteen probabilities characterizing the ground state, $N = 1000$ time steps are needed. This results in a total of roughly 8000 single-qubit gates and 18,000 two-qubit gates.

Since two-qubit gate errors dominate in quantum devices, their impact is critical. According to Figure 15, the IONQ Forte device has a two-qubit gate error rate of $p = 1.530 \times 10^{-2}$ [2]. The probability of at least one gate failure during the entire sequence can be calculated as:

$$P_{\text{failure}} = 1 - (1 - p)^{18000}. \tag{17}$$

Given the high number of gates, this probability is effectively 1, indicating that nearly every execution will contain errors. Reducing the number of time steps N might seem like a solution, but doing so would require using Trotter-Suzuki approximations, which increase the number of two-qubit gates per time step.

As a result, the cumulative gate errors would still prevent accurate adiabatic simulation toward the ground state. Under current NISQ hardware limitations, this makes adiabatic evolution impractical for achieving the desired level of accuracy.

Part 3 : A simpler path to the ground state

Build a numerical program as outlined above, which contains arbitrary angle parameters and qubit pairs for XY-mixers. Experiment with the XY-mixers to obtain angle parameters and qubit pairs that rotate the final state so that its probabilities match the ground state probabilities of H_{sg}. This problem can be solved using three XY-mixers.

I used an ansatz initialized to the $|1110\rangle$ state, followed by three XY-mixers acting on the qubit pairs (qubit-0 and qubit-1), (qubit-0 and qubit-2), and (qubit-0 and qubit-3). The following code performs these operations:

```
def ansatz(self, parameters):
  if len(self.index) != len(parameters):
    raise ValueError("Lengh should be matched")

  state = np.zeros(16, dtype=np.complex64)
  state[14] = 1

  operation = [self._XY_mixer(pair, parameter)
               for pair, parameter
               in zip(self.index, parameters)]
  for op in operation:
    state = op @ state

  return state
```

After constructing the ansatz, the probabilities of the resulting state are compared with the probability amplitudes of the exact ground state obtained using `np.linalg.eigvalsh`. The following cost function calculates the L2-norm between the two:

```
def cost(self, parameters):
  state = self.ansatz(parameters)
  amps = np.abs(state)**2
  cost = np.linalg.norm(amps - self.target)

  return cost
```

Finally, we use the `scipy.optimize.minimize` method to optimize the circuit parameters for the XY-mixer. This optimization routine adjusts the angles to minimize the cost function, ensuring the ansatz approximates the ground state as closely as possible:

```
def optimize(self):
  random_para = np.random.rand(3)*2*np.pi
  res = scipy.optimize.minimize(self.cost,
              random_para, method="L-BFGS-B")

  return res
```

Convert the numerical program into a quantum circuit. Verify that the output of this quantum circuit generates the correct probabilities for the ground state of H_{sg}.

Noting that $\left[\sigma_x^i\sigma_x^j, \sigma_y^i\sigma_y^j\right] = 0$ the XY-mixer operator can be decomposed as follows:

$$e^{-i\frac{\theta}{2}\left(\sigma_x^i\sigma_x^j+\sigma_y^i\sigma_y^j\right)} = e^{-i\frac{\theta}{2}\sigma_x^i\sigma_x^j}e^{-i\frac{\theta}{2}\sigma_y^i\sigma_y^j} \tag{18}$$

This decomposition corresponds to a combination of the RXX gate and the RYY gate, respectively.

To construct the quantum circuit: First, Apply X gates to qubits 1, 2, and 3 to initialize the system in the $|1110\rangle$ state. Second, Apply parameterized RXX and RYY gates on the qubit pairs (0,1), (0,2), and (0,3), corresponding to the XY-mixer operations. This quantum circuit prepares a state that closely generates a probability distribution similar to the exact ground state. The implementation is as follows:

```
def circuit_without_phase(self, parameters):
  qc = qiskit.QuantumCircuit(self.num_qubits)
  for i in range(1,self.num_qubits):
    qc.x(i)
  for pair, parameter in zip(self.index, parameters):
    qc.rxx(parameter, pair[0], pair[1])
    qc.ryy(parameter, pair[0], pair[1])

  return qc
```

Check the phases of the non-zero components of the quantum circuit created in 3.b. Are they all real? If so, you can skip this step. Otherwise, find circuit elements that you can add to the end of your circuit so that all non-zero computational basis states have the same phase.

The exact ground state of H_{sg} has real coefficients with signs $(+,+,+,-)$ corresponding to the basis states $|0111\rangle, |1011\rangle, |1101\rangle, |1110\rangle$. However, the quantum circuit constructed above generates a state where the coefficient of $ket1110$ is real, while the other coefficients are purely imaginary. Specifically, the state produced by the circuit can be expressed as:

$$\begin{aligned} & e^{-i\frac{\theta_3}{2}\left(\sigma_x^0\sigma_x^3+\sigma_y^0\sigma_y^3\right)}e^{-i\frac{\theta_2}{2}\left(\sigma_x^0\sigma_x^2+\sigma_y^0\sigma_y^2\right)}e^{-i\frac{\theta_1}{2}\left(\sigma_x^0\sigma_x^1+\sigma_y^0\sigma_y^1\right)}|1110\rangle \\ = & -i\cos\theta_1\cos\theta_2\sin\theta_3|0111\rangle + i\cos\theta_1\sin\theta_2|1011\rangle \\ & -i\sin\theta_1|1101\rangle - \cos\theta_1\cos\theta_2\cos\theta_3|1110\rangle \end{aligned}$$

To convert the imaginary coefficients to real values, apply an $R_z\left(\frac{\pi}{2}\right)$ gate to qubit 0. This operation rotates the phases of the basis states such that all imaginary components become real. After correcting the phases, the signs of the coefficients need to match the target $(+,+,+,-)$ or $(-,-,-,+)$. An $R_z(\pi)$ gate is applied conditionally:

- If a basis state has the same sign as $|1110\rangle$, no further operation is needed.
- Otherwise, apply an $R_z(\pi)$ gate to the corresponding qubit to flip the sign appropriately.

The following code implements the phase adjustment and sign correction:

```
def circuit(self, parameters):
  qc = qiskit.QuantumCircuit(self.num_qubits)
  for i in range(1,self.num_qubits):
    qc.x(i)
  for pair, parameter in zip(self.index, parameters):
    qc.rxx(parameter, pair[0], pair[1])
    qc.ryy(parameter, pair[0], pair[1])

  qc.rz(np.pi/2, 0)
  temp = self.ansatz(parameters)[[7,11,13,14]]

  temp[0:3] *= (1j if temp[3] > 0 else -1j)
  temp = (temp > 0)
  for i in range(3):
    qc.rz(np.pi, 3-i) if temp[i] else None
```

```
        self.qc = qc

        return qc
```

This implementation ensures the quantum circuit generates the correct ground state of H_{sg}, with all non-zero components having the correct phase and sign.

Use your quantum circuit to measure the expectation value of the ground state of H_{sg} . You will have to repeat your circuit three times - once to measure in each of the X, Y and Z computational bases. Each term of H_{sg} can be measured in one of the three computational bases. Sum the contributions from the three bases and confirm that the summed energy is equal to the ground state eigenvalue

The expectation value of the system Hamiltonian H_{sg} can be computed as the sum of the expectation values of its individual components. To calculate the expectation value of operators such as σ_z^i and $\sigma_z^i\sigma_z^j$, we decompose the Hamiltonian into measurable components and measure each component separately in the appropriate computational basis.

First, σ_z^i can be decomposed into:

$$\begin{aligned}\sigma_z^i &= I_2^{\otimes 3-i} \otimes \sigma_z \otimes I_2^{\otimes i} \\ &= (|0\rangle\langle 0| + |1\rangle\langle 1|)^{\otimes 3-i} \otimes (|0\rangle\langle 0| - |1\rangle\langle 1|) \otimes (|0\rangle\langle 0| + |1\rangle\langle 1|)^{\otimes i}\end{aligned}$$

where I_2 is 2×2 idenity matrix. Then the expectation value of σ_z^i can be expressed as follows:

$$\langle \sigma_z^i \rangle = \sum_{(b_i=0)} m_b - \sum_{(b_i=1)} m_b \tag{19}$$

where $b = b_3b_2b_1b_0$ is the bit string, and $m_b = |\langle b|\psi\rangle|^2$ is the probability amplitude of the bitstring b in the given state $|\psi\rangle$. The expectation value is obtained by summing over the measurement outcomes weighted by ± 1, depending on whether the corresponding qubit is in the state $|0\rangle$ or $|1\rangle$.

Similarly, for the two-qubit operator, $\sigma_z^i\sigma_z^j$ is decomposed into:

$$\begin{aligned}\sigma_z^i\sigma_z^j &= I_2^{\otimes 3-j} \otimes \sigma_z \otimes I_2^{\otimes i-j-1} \otimes \sigma_z \otimes I_2^{\otimes i} \\ &= (|0\rangle\langle 0| + |1\rangle\langle 1|)^{\otimes 3-i} \otimes (|0\rangle\langle 0| - |1\rangle\langle 1|) \\ &\quad \otimes (|0\rangle\langle 0| + |1\rangle\langle 1|)^{\otimes i-j-1} \\ &\quad \otimes (|0\rangle\langle 0| - |1\rangle\langle 1|) \\ &\quad \otimes (|0\rangle\langle 0| + |1\rangle\langle 1|)^{\otimes i}\end{aligned}$$

where $i > j$ is supposed without loss of generality. Then, the expectation value of $\sigma_z^i \sigma_z^j$ is computed by considering whether the product of the states of qubits i amd j is $|00\rangle$ or $|11\rangle$(yielding $+1$) or $|01\rangle$ or $|10\rangle$(yielding -1):

$$\left\langle \sigma_z^i \sigma_z^j \right\rangle = \sum_{(b_i \oplus b_j = 0)} m_b - \sum_{(b_i \oplus b_j = 1)} m_b \tag{20}$$

The following code performs the expectation value extraction:

```
def _calculate_expectation(self, basis, i, j=None):
  expectation_value = 0
  measurement_outcome = self.measurement_outcome[basis]

  for bitstring, probability in measurement_outcome.items():
    bit_i = (int(bitstring, 2) >> i) & 1
    if j is not None:
      bit_j = (int(bitstring, 2) >> j) & 1
      sign = 1 if (bit_i ^ bit_j) == 0 else -1
    else:
      sign = 1 if bit_i == 0 else -1

    expectation_value += sign * probability

  return expectation_value
```

The expectation values of operators like $\left\langle \sigma_x^i \sigma_x^j \right\rangle$ and $\left\langle \sigma_y^i \sigma_y^j \right\rangle$ calculated similarly, but measurements are performed in the X and Y basis, respectively. Generally, the measurement is performed on z basis, which are described as POVM set, $\{|0000\rangle\langle 0000|, |0001\rangle\langle 0001|, \cdots, |1111\rangle\langle 1111|\}$. By applying additional unitary operation before the measurement, we can make it as if we are performing measurement on a different basis. Then, the standard Z-basis measurement is modified by applying suitable unitary transformations before the measurement:

- For the X-basis, apply a Hadamard gate to each qubit, before the measurement.
- For the Y-basis, apply a sequence of S-gate and Hadamard gate to each qubit, before the measurement.
- For the Z-basis, no additional gates are required.

The following code generates quantum circuits to perform these measurements:

```
def perform_measure(self, num_shots=10000):
  qc_x = self.qc.copy()
  qc_y = self.qc.copy()
  qc_z = self.qc.copy()

  for i in range(self.num_qubits):
    qc_x.h(i)
    qc_y.s(i)
    qc_y.h(i)

  qc_x.measure_all()
  qc_y.measure_all()
  qc_z.measure_all()

  outcome_x = MeasurementResultHandler(backend
                .run(qc_x, shots=num_shots).result()
                .data()["counts"],
                self.num_qubits, num_shots)
  outcome_y = MeasurementResultHandler(backend
                .run(qc_y, shots=num_shots).result()
                .data()["counts"],
                self.num_qubits, num_shots)
  outcome_z = MeasurementResultHandler(backend
                .run(qc_z, shots=num_shots).result()
                .data()["counts"],
                self.num_qubits, num_shots)

  self.measurement_outcome = {
    "x" : outcome_x,
    "y" : outcome_y,
    "z" : outcome_z,
  }

  return self.measurement_outcome
```

where `MeasurementResultHandler` processes the measurement results and fills any missing outcomes with zero probabilities.

```
def MeasurementResultHandler(result:dict, num_qubits:int, num_shots:int):
        new_result = {}
        for i in range(2**num_qubits):
                temp = format(i,"#b")[2:]
```

```
                key = "".join(["0"
          for _ in
          range(num_qubits - len(temp))])
          + temp
                new_result[key] = np.array(
                  result[format(i,"#x")])
                  /num_shots
                  if result.get(format(i,"#x"))
                  else np.array(0)
        return new_result
```

Finally, The expectation value of H_{sg} is calculated using the following code:

```
def energy(self):
  energy = 0
  combinations = itertools.combinations([i
                 for i in range(self.num_qubits)], 2)

  for coefficient, pair in zip(self.coefficients1, combinations):
    energy += coefficient * self._calculate_expectation(
                                  "x", pair[0], pair[1])
    energy += coefficient * self._calculate_expectation(
                                  "y", pair[0], pair[1])
    energy += coefficient * self._calculate_expectation(
                                  "z", pair[0], pair[1])

  for idx, coefficient in enumerate(self.coefficients2):
    energy += coefficient * self._calculate_expectation(
                                  "z", idx)

  return energy
```

I conducted a simulation to extract the ground state energy from three circuit executions, varying the number of measurement shots from 10^3 to 10^7. For each shot count, 100 simulations were performed to analyze the accuracy and consistency of the measured energy.

The figure illustrates how the accuracy of the measured energy $\langle H_{sg} \rangle$ improves with an increasing number of measurement shots. On the left, the measured energy (blue markers) converges toward the exact energy (orange line) as the total number of shots increases. The shaded region denotes the error margin around the measured energy, which progressively narrows, indicating reduced error.

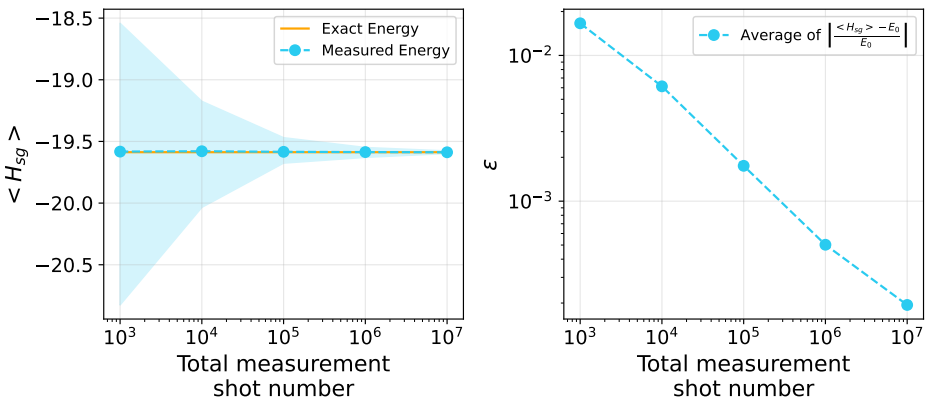

Figure 16: Comparison of the measured energy and its accuracy as a function of total measurement shot number. (Left) The measured energy $\langle H_{sg} \rangle$(blue markers) approaches the exact ground state energy (orange line) as the total number of shots increases. The shaded area represents the error margin around the measured energy. (Right) The average relative error ϵ of the measured energy decreases as the total number of shots increases. The relative error is calculated as $\left|\frac{\langle H_{sg} \rangle - E_0}{E_0}\right|$.

On the right, the average relative error ϵ of the measured energy from the exact ground state energy E_0 is plotted against the total shot number A clear trend emerges: the average relative error decreases sharply as the shot count increases.

As shown in Figure 16, with more than 10^6 shots, therefore, the average relative error falls below 10^{-3}, enabling an accurate extraction of the ground state energy.

References

[1] Dominic W. Berry, Graeme Ahokas, Richard Cleve, and Barry C. Sanders. Efficient quantum algorithms for simulating sparse hamiltonians. *Communications in Mathematical Physics*, 270(2):359–371, December 2006.

[2] Jwo-Sy Chen, Erik Nielsen, Matthew Ebert, Volkan Inlek, Kenneth Wright, Vandiver Chaplin, Andrii Maksymov, Eduardo Páez, Amrit Poudel, Peter Maunz, and John Gamble. Benchmarking a trapped-ion quantum computer with 30 qubits, 2024.

[3] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. Quantum adiabatic evolution algorithms versus simulated annealing, 2002.

[4] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, Joshua Lapan, Andrew Lundgren, and Daniel Preda. A quantum adiabatic evolution algorithm applied to random instances of an np-complete problem. *Science*, 292(5516):472–475, 2001.

[5] Hans De Raedt. *Computer simulation of quantum phenomena in nanoscale devices*, pages 107 – 146. University of Groningen, The Zernike Institute for Advanced Materials, 1996.

[6] Peter Wittek and Fernando M. Cucchietti. A second-order distributed trotter–suzuki solver with a hybrid cpu–gpu kernel. *Computer Physics Communications*, 184(4):1165–1171, April 2013.